Universidad de La Laguna

Grado en Ingeniería Informática

Computabilidad y Algoritmia

Práctica 7

Diseño y simplificación de gramáticas en JFLAP

Luis Eduardo Martín Hernández
alu0101631134@ull.edu.es

6. Ejercicios

1. **Diseñar una gramática independiente del contexto que genere el lenguaje $L = \{a^n b^n \mid n \geq 0\}$**

Se considera una producción sobre sí misma de S que tenga en cuenta cualquier cantidad de a's al inicio igual a la cantidad de b's al final. Se detendrá en el caso base que sería ε.

$S \rightarrow aSb \mid \varepsilon$

Input	Result
aabb	Accept
ab	Accept
	Accept
aaaabbbb	Accept
aaaaaabbbbbb	Accept
b	Reject
a	Reject
ba	Reject
bbbaa	Reject
aab	Reject

A continuación es momento de simplificar la gramática:

1. Eliminación de producciones vacías:

Como no hay ninguna producción vacía a parte de S → ε, es decir, la cadena vacía pertenece a la gramática. El resultado del algoritmo es el anterior.

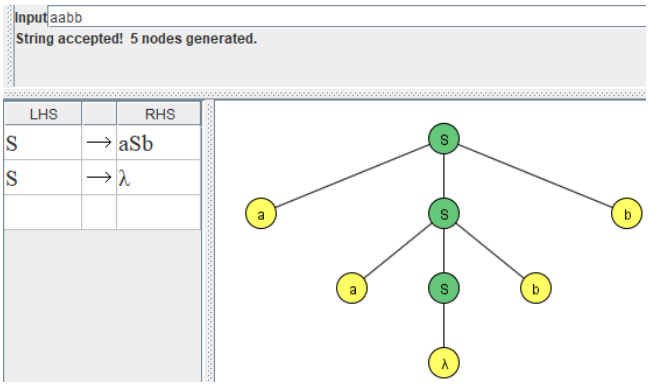
2. Eliminación de producciones unitarias:

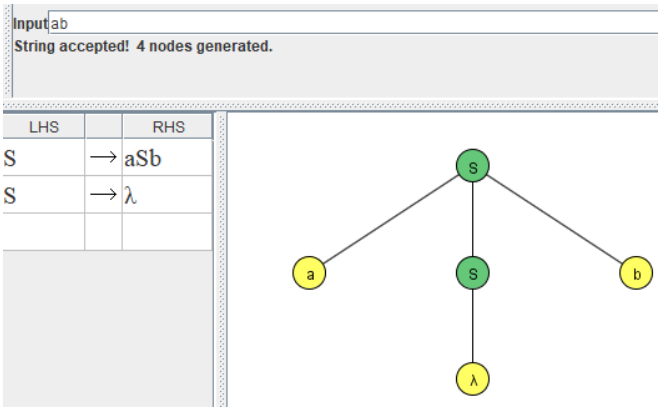
No existe ninguna producción unitaria

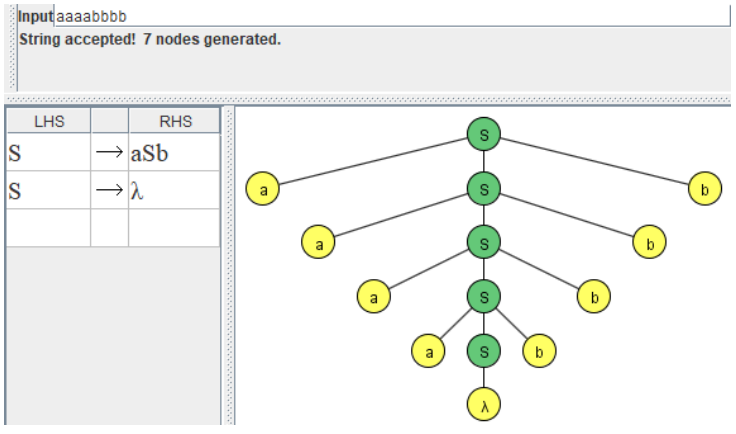
3. Eliminación de símbolos y producciones inútiles

Etapa 1:
V' = ∅
debido a la producción S → ε se añade S a V': V' = V' ∪ {S}; V' = {S}
Como V = V' no se elimina ningún no terminal

Etapa 2:
J = {S}, V' = {S}, Σ' = ∅
Se extrae S de J
Se añaden a Σ' todos los no terminales de las producciones a partir de S:
Σ' = {a, b}
No se encuentra ningún no terminal nuevo.
Como V' = V y Σ' = Σ:
El resultado es el mismo al anterior.







2. **Diseñar una gramática independiente del contexto que genere el lenguaje $L = \{a^n b^m \mid n, m \geq 0, n \neq m\}$**

Se realiza una producción sobre sí misma de S en la que siempre se tiene la misma cantidad de a y b hasta que se elige otra producción y cambia el número de uno y solo uno de estos símbolos.

$S \rightarrow aA \mid bB \mid aSb \mid a \mid b$
$A \rightarrow aA \mid a$
$B \rightarrow bB \mid b$

Input	Result
aab	Accept
aabbb	Accept
abb	Accept
b	Accept
a	Accept
	Reject
ab	Reject
aabb	Reject
ba	Reject
bba	Reject

A continuación es momento de simplificar la gramática:

1. Eliminación de producciones vacías:

Como no hay ninguna producción vacía. El resultado del algoritmo es el anterior.

2. Eliminación de producciones unitarias:

No existe ninguna producción unitaria

3. Eliminación de símbolos y producciones inútiles

Etapa 1:
V' = ∅
debido a la producción $S \rightarrow \varepsilon$ se añade S a V': V' = V' ∪ {S}; V' = {S}
debido a la producción $A \rightarrow a$ se añade A a V': V' = V' ∪ {A}; V' = {S, A}
debido a la producción $B \rightarrow b$ se añade B a V': V' = V' ∪ {B}; V' = {S, A, B}
Como V = V' no se elimina ningún no terminal

Etapa 2:
J = {S}, V' = {S}, Σ' = ∅
Se extrae S de J
Se añaden a Σ' todos los no terminales de las producciones a partir de S:
Σ' = {a, b}
Se encuentran los no terminales A y B por lo que:
J = J ∪ {A, B}

V’ = V’ ∪ {A, B}
Se repite el proceso hasta que J es vacío pero no se añade ningún terminal ni ningún no terminal. V’ = V y Σ’ = Σ.
El resultado es el mismo al anterior.

LHS		RHS
S	→	aSb
S	→	aA
S	→	bB
S	→	a
S	→	b
A	→	aA
B	→	bB
A	→	a
B	→	b

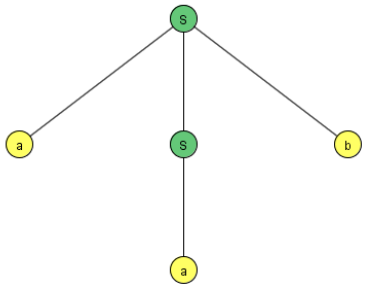
Convert Selected | Do All | What's Left? | Export

Welcome to the Chomsky converter.
5 production(s) must be converted.

LHS		RHS
S	→	aSb
S	→	aA
S	→	bB
S	→	a
S	→	b
A	→	aA
B	→	bB
A	→	a
B	→	b

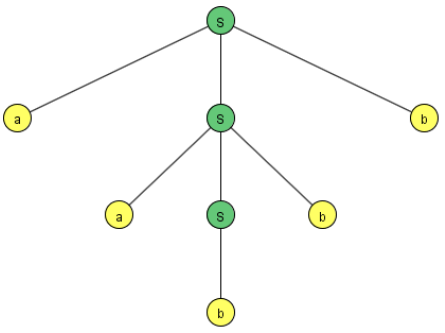
Input aab
String accepted! 5 nodes generated.

LHS		RHS
S	→	aSb
S	→	aA
S	→	bB
S	→	a
S	→	b
A	→	aA
B	→	bB
A	→	a
B	→	b



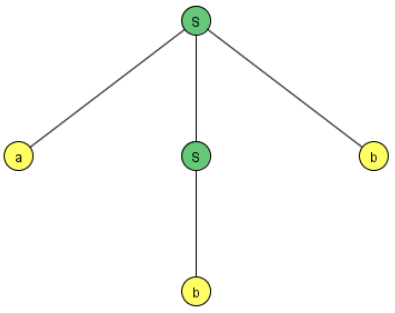
Input aabbb
String accepted! 10 nodes generated.

LHS		RHS
S	→	aSb
S	→	aA
S	→	bB
S	→	a
S	→	b
A	→	aA
B	→	bB
A	→	a
B	→	b



Input abb
String accepted! 5 nodes generated.

LHS		RHS
S	→	aSb
S	→	aA
S	→	bB
S	→	a
S	→	b
A	→	aA
B	→	bB
A	→	a
B	→	b



3. **Diseñar una gramática independiente del contexto para el lenguaje $L = \{ww^i \mid w \in \{a, b\}*\}$**

La gramática construye palíndromos añadiendo el mismo símbolo al inicio y al final de la cadena en cada paso. Esto a partir de dos producciones recursivas.

$$S \rightarrow aSa \mid bSb \mid a \mid b \mid \varepsilon$$

Input	Result
	Accept
abba	Accept
bab	Accept
bb	Accept
aa	Accept
aab	Reject
ab	Reject
a	Accept
b	Accept
bba	Reject

A continuación es momento de simplificar la gramática:

1. Eliminación de producciones vacías:

Como no hay ninguna producción vacía a parte de $S \rightarrow \varepsilon$, es decir, la cadena vacía pertenece a la gramática. El resultado del algoritmo es el anterior.

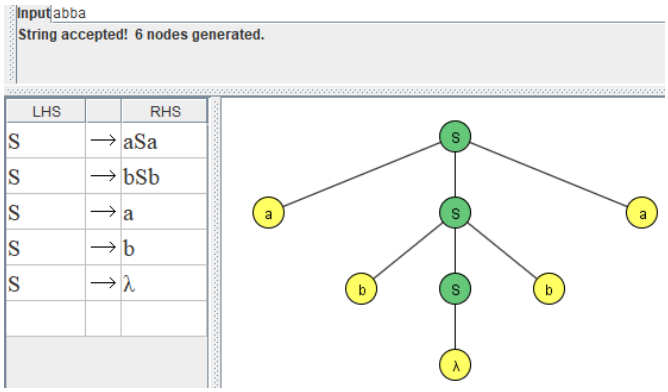
2. Eliminación de producciones unitarias:

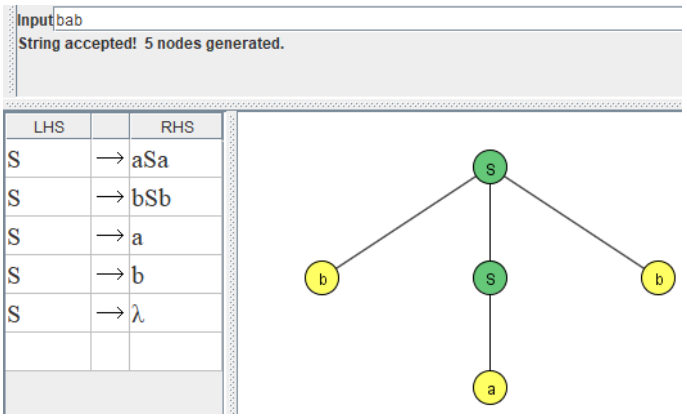
No existe ninguna producción unitaria

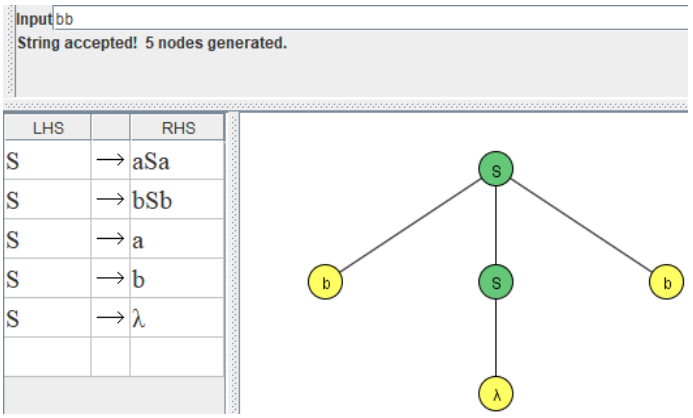
3. Eliminación de símbolos y producciones inútiles

Etapa 1:
$V' = \varnothing$
debido a la producción $S \rightarrow a$ se añade S a V': $V' = V' \cup \{S\}$; $V' = \{S\}$
Como V = V' no se elimina ningún no terminal

Etapa 2:
$J = \{S\}$, $V' = \{S\}$, $\Sigma' = \varnothing$
Se extrae S de J
Se añaden a Σ' todos los no terminales de las producciones a partir de S:
$\Sigma' = \{a, b\}$
No se encuentra ningún no terminal nuevo.
Como V' = V y Σ' = Σ:
El resultado es el mismo al anterior.







4. **Diseñar una gramática independiente del contexto que genere el lenguaje $L = \{a^n b^m c^n \mid n \geq 0, m \text{ impar}\}$**

Funciona igual que el ejercicio 1 con la diferencia de que tenemos que añadir una cantidad de b's impar en el medio de la cadena.

S → aSc | bB

B → bbB | ε

Input	Result
abc	Accept
aaabbbccc	Accept
abbbc	Accept
aabbbbbcc	Accept
b	Accept
	Reject
a	Reject
ac	Reject
abbc	Reject
bac	Reject

A continuación es momento de simplificar la gramática:

1. Eliminación de producciones vacías:

H = ∅
La producción B → ε es una producción por lo que H = H ∪ {B}
Como existe la producción S → bB : H = H ∪ {S}

Elimino todas las producciones vacías, en este caso B → ε.

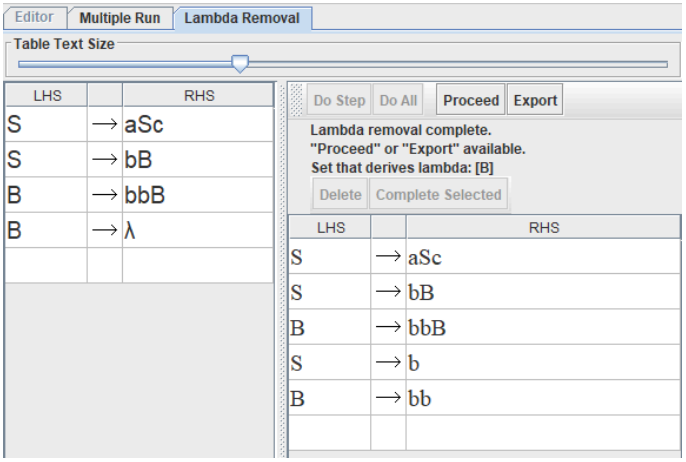
Para todos los elementos elementos de H:
Se eliminan los símbolos no terminales o terminales de las producciones y se generan nuevos dependiendo de si son anulables o no anulables.

Obtenemos lo siguiente:

S → aSc | b(B∨ε)
B → bbB | bb

Resultado:

S → aSc | bB | b
B → bbB | bb



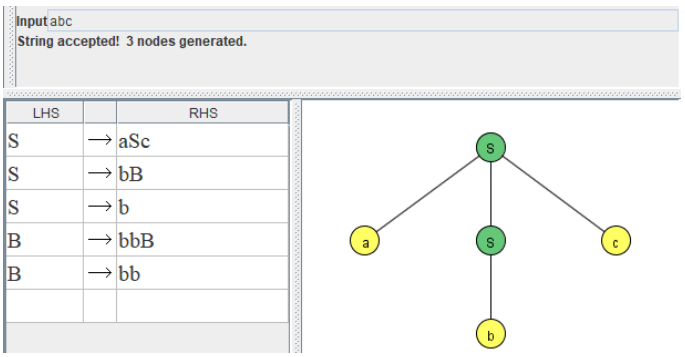
2. Eliminación de producciones unitarias:

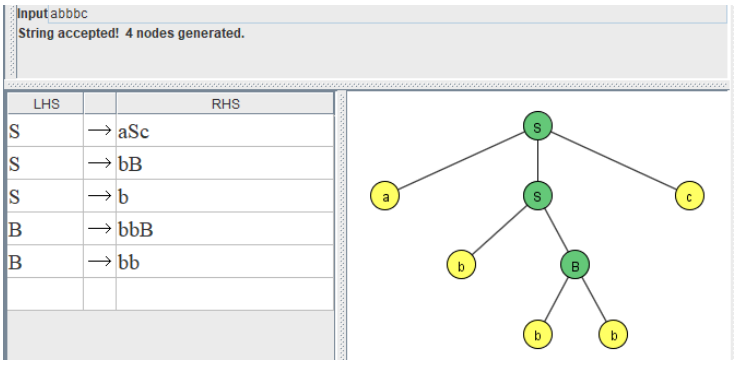
No existe ninguna producción unitaria

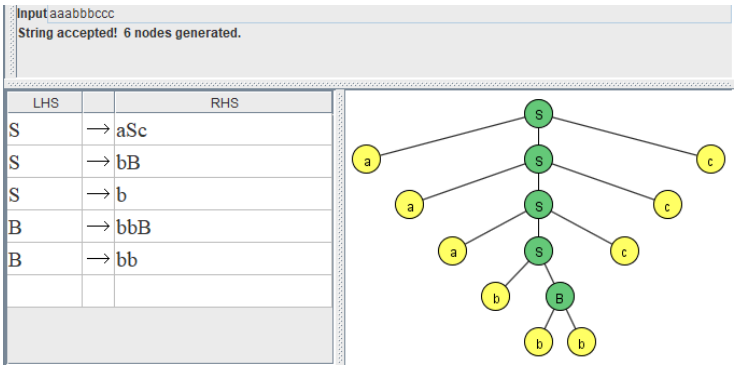
3. Eliminación de símbolos y producciones inútiles

Etapa 1:
V' = ∅
debido a la producción S → b se añade S a V': V' = V' ∪ {S}
debido a la producción B → bb se añade B V': V' = V' ∪ {S, B}
Como V = V' no se elimina ningún no terminal

Etapa 2:
J = {S}, V' = {S}, Σ' = ∅
Se extrae S de J
Se añaden a Σ' todos los no terminales de las producciones a partir de S:
Σ' = {a, b, c}
Como B no pertenece a J: J = J ∪ {B}; J = {B}
Y se añade B a V': V' = V' ∪ {B}; V' = {S, B}
Se repite el proceso pero como V = V' y Σ = Σ', J se queda vacía y termina el algoritmo
El resultado es el mismo al anterior.







5. **Diseñar una gramática independiente del contexto que genere el lenguaje $L = \{a^n b^m c^m d^n \mid n, m \geq 1\}$**

Es necesario una primera producción de S sobre sí misma para que añada todas las a y d al inicio y al final de la cadena, a continuación una llamada a A que realiza lo mismo con b y c y una última producción terminal bc que completa la cadena.

$S \rightarrow aSd \mid aAd$

$A \rightarrow bAc \mid bc$

Input	Result
abcd	Accept
aabcdd	Accept
abbccd	Accept
aabbccdd	Accept
aaabcddd	Accept
	Reject
a	Reject
c	Reject
d	Reject
aabbcd	Reject

A continuación es momento de simplificar la gramática:

1. Eliminación de producciones vacías:

Como no hay ninguna producción vacía. El resultado del algoritmo es el anterior.

2. Eliminación de producciones unitarias:

No existe ninguna producción unitaria

3. Eliminación de símbolos y producciones inútiles

Etapa 1:
V' = ∅
debido a la producción A → bc se añade A a V': V' = V' ∪ {A}; V' = {A}
debido a la producción S → aAd y que A pertenece a V', añadimos S a V': V' = V' ∪ {S}
Como V = V' no se elimina ningún no terminal

Etapa 2:
J = {S}, V' = {S}, Σ' = ∅
Se extrae S de J
Se añaden a Σ' todos los no terminales de las producciones a partir de S:
Σ' = {a, d}
Se encuentra el no terminal A:
J = J ∪ {A}
V' = V' ∪ {A}
Se repite el proceso
Se extrae A de J
Se añaden a Σ' todos los no terminales de las producciones a partir de A:
Σ' = {a,b, c, d}
No se encuentran más no terminales
Y se acaba el bucle
Como V' = V y Σ' = Σ.
El resultado es el mismo al anterior.

LHS		RHS
S	→	aSd
S	→	aAd
A	→	bAc
A	→	bc

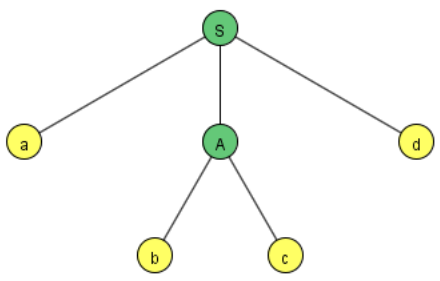
Convert Selected | Do All | What's Left? | Export

Welcome to the Chomsky converter.
4 production(s) must be converted.

LHS		RHS
S	→	aSd
S	→	aAd
A	→	bAc
A	→	bc

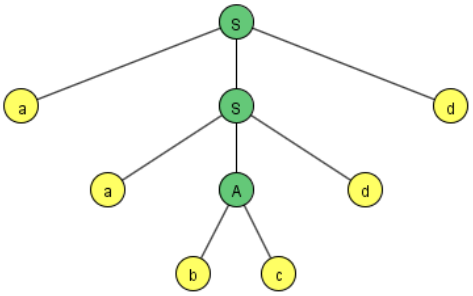
Input abcd
String accepted! 4 nodes generated.

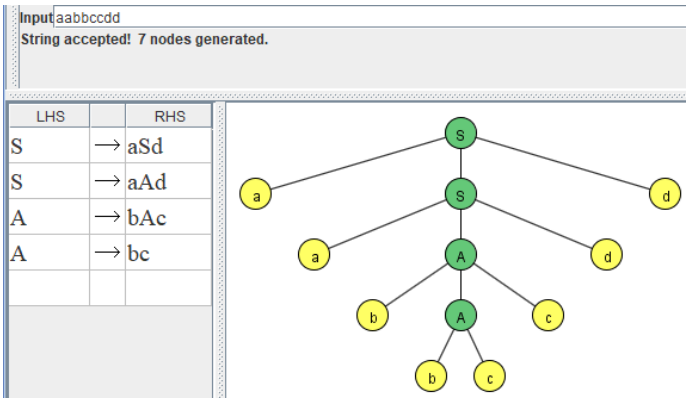
LHS		RHS
S	→	aSd
S	→	aAd
A	→	bAc
A	→	bc



Input aabcdd
String accepted! 6 nodes generated.

LHS		RHS
S	→	aSd
S	→	aAd
A	→	bAc
A	→	bc





6. **Diseñar una gramática independiente del contexto que genere el lenguaje $L = \{a^n b^m \mid n > m \geq 0\}$**

Para obtener el resultado es necesario una producción como la del ejercicio 1, otras 3 producciones para tener una cantidad e a's mayor que de b's y una última que contempla la posibilidad de w = a.

$S \rightarrow aSb \mid aA \mid a$
$A \rightarrow aA \mid a$

Input	Result
a	Accept
aaa	Accept
aab	Accept
aaabb	Accept
aaaaab	Accept
	Reject
b	Reject
ab	Reject
aabb	Reject
ba	Reject

A continuación es momento de simplificar la gramática:

1. Eliminación de producciones vacías:

Como no hay ninguna producción vacía. El resultado del algoritmo es el anterior.

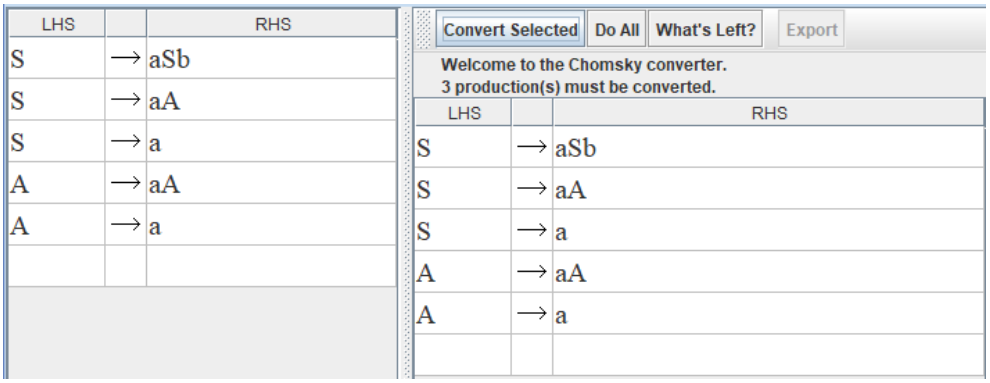
2. Eliminación de producciones unitarias:

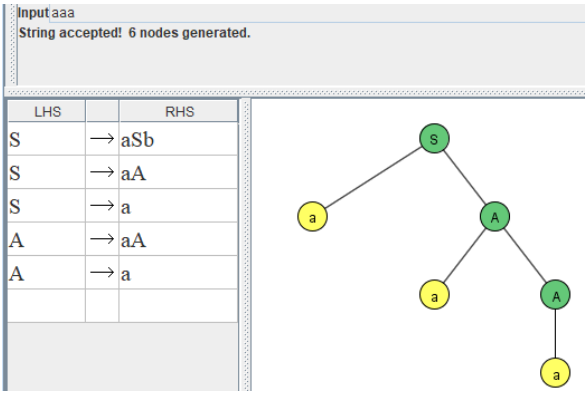
No existe ninguna producción unitaria

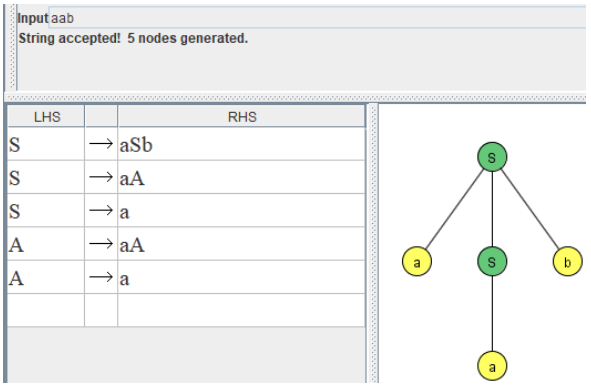
3. Eliminación de símbolos y producciones inútiles

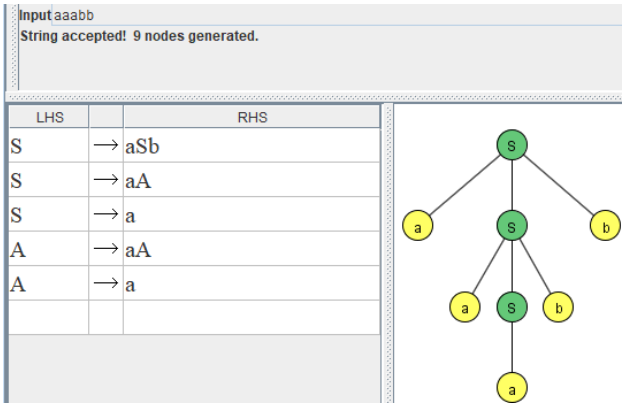
Etapa 1:
V' = ∅
debido a la producción S → a se añade S a V': V' = V' ∪ {S}; V' = {S}
debido a la producción A → a se añade A a V': V' = V' ∪ {A}; V' = {S, A}

Como V = V’ no se elimina ningún no terminal
Etapa 2:
J = {S}, V’ = {S}, Σ’ = ∅
Se extrae S de J
Se añaden a Σ’ todos los no terminales de las producciones a partir de S:
Σ’ = {a, b}
Se encuentra el no terminal A:
J = J ∪ {A}
V’ = V’ ∪ {A}
Se repite el proceso
No se encuentran terminales nuevos
No se encuentran más no terminales
Y se acaba el bucle
Como V’ = V y Σ’ = Σ.
El resultado es el mismo al anterior.









7. **Diseñar una gramática independiente del contexto que genere el lenguaje $L = \{a^i b^j c^{i+j} \mid i, j \geq 1, i + j \text{ par}\}$**

Se distinguen dos opciones en un principio, cantidad de a's par y cantidad de a's impar (teniendo en cuenta en todo momento que hay que añadir la misma cantidad de a y b que de c's). En la primera producción de A (asignada a la cantidad de a's impar) permite añadir una cantidad de a's arbitrarias de dos en dos. En la segunda y tercera producción se añade una única b y para que la cantidad final de b's sea impar. Las producciones de B son las asignadas a la cantidad de a's pares por lo que en su segunda y tercera producción se añaden una cantidad par de b's. Por último, la producción C solo ofrece añadir una cantidad arbitraria par de b's.

S → aAc | aaBcc
A → aaAcc | bCc | bc
B → aaBcc | bbCcc | bbcc
C → bbCcc | bbcc

Input	Result
abcc	Accept
aaabcccc	Accept
aabbcccc	Accept
abbbcccc	Accept
aaabbbcccccc	Accept
	Reject
abc	Reject
abbccc	Reject
abccc	Reject
ab	Reject

A continuación es momento de simplificar la gramática:

1. Eliminación de producciones vacías:

Como no hay ninguna producción vacía. El resultado del algoritmo es el anterior.

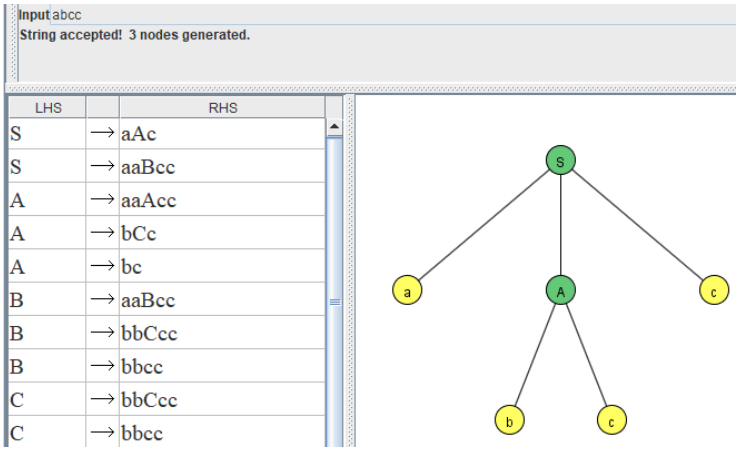
2. Eliminación de producciones unitarias:

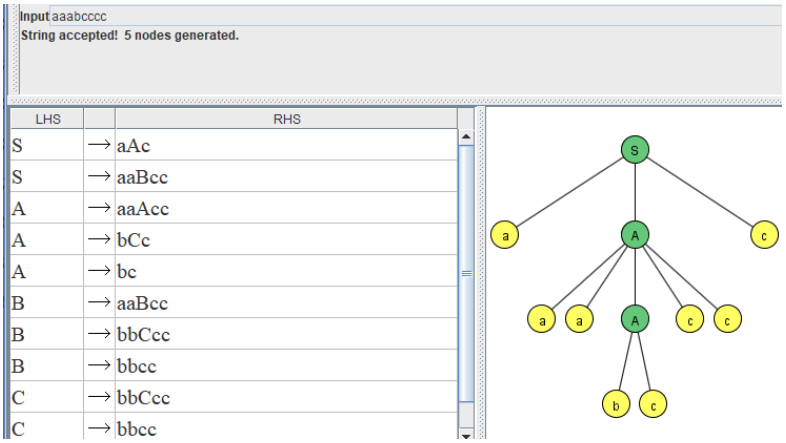
No existe ninguna producción unitaria

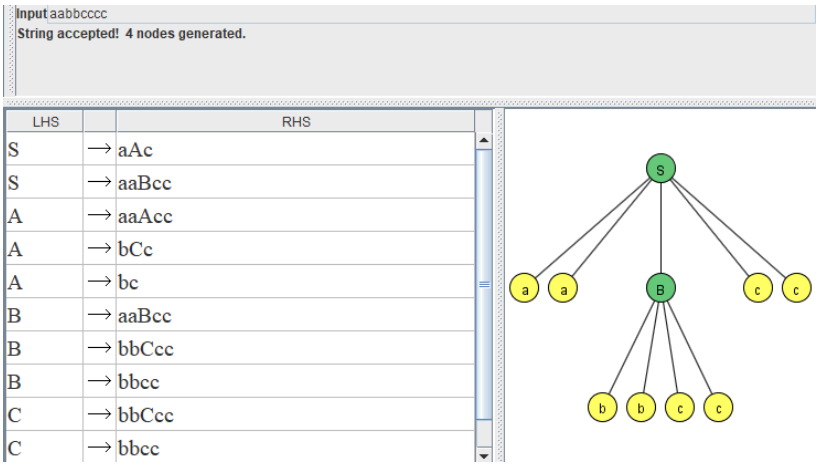
3. Eliminación de símbolos y producciones inútiles

Etapa 1:
V' = ∅
debido a la producción A → bc se añade A a V': V' = V' ∪ {A}; V' = {A}
debido a la producción B → bbcc se añade B a V': V' = V' ∪ {B}; V' = {A, B}
debido a la producción C → bbcc se añade C a V': V' = V' ∪ {C}; V' = {A, B, C}
debido a la producción S → aAc y A pertenece a V': V' = V' ∪ {S}; V' = {S, A, B, C}
Como V = V' no se elimina ningún no terminal
Etapa 2:
J = {S}, V' = {S}, Σ' = ∅

Se extrae S de J
Se añaden a Σ' todos los no terminales de las producciones a partir de S
Σ' = {a, c}
Se añade A a J: J = J ∪ {A}
Se repite el bucle extrayendo A y se obtienen los terminales que aparecen en las producciones de A
Σ' = {a, b, c}
Se añade B a J y se repite el proceso continuamente hasta que la J quede vacía
Al final se obtiene que Σ' = Σ y que V' = V' por lo que el resultado es el mismo al anterior.







8. **Diseñar una gramática independiente del contexto que genere el lenguaje de las expresiones booleanas con los operadores AND, OR, y NOT, usando paréntesis para agrupar. Ejemplos: “(true AND false)”, “NOT (true OR false)”, “true AND (false OR true)”.**

Para obtener una gramática que represente las operaciones booleanas es necesario usar producciones recursivas sobre S tales que solo sean terminales con las cadenas true y false.

$$S \rightarrow S \vee S \mid S \wedge S \mid (S) \mid \sim S \mid true \mid false$$

Input	Result
true ∨ false	Accept
true ∨ (false ∧ ~true)	Accept
~false	Accept
(true)	Accept
~(true)	Accept
	Reject
false ~true	Reject
false ∨	Reject
true ∧	Reject
true false	Reject

A continuación es momento de simplificar la gramática:

1. Eliminación de producciones vacías:

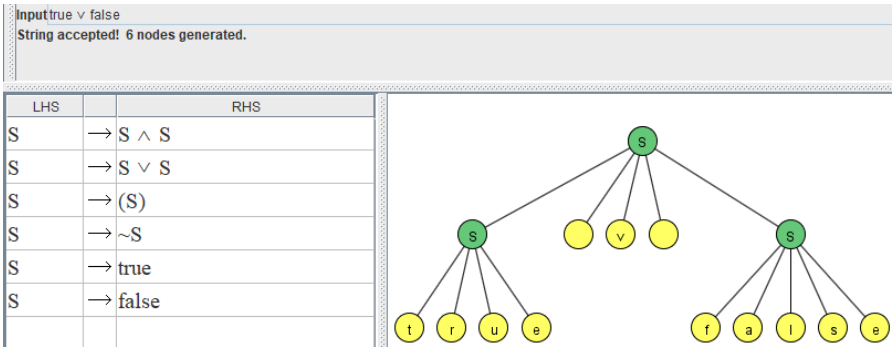
Como no hay ninguna producción vacía. El resultado del algoritmo es el anterior.

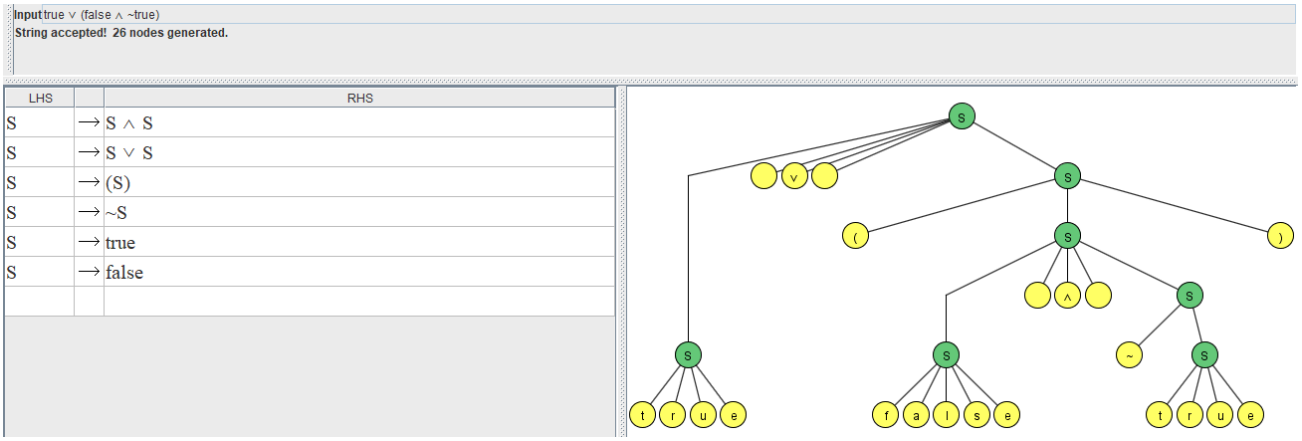
2. Eliminación de producciones unitarias:

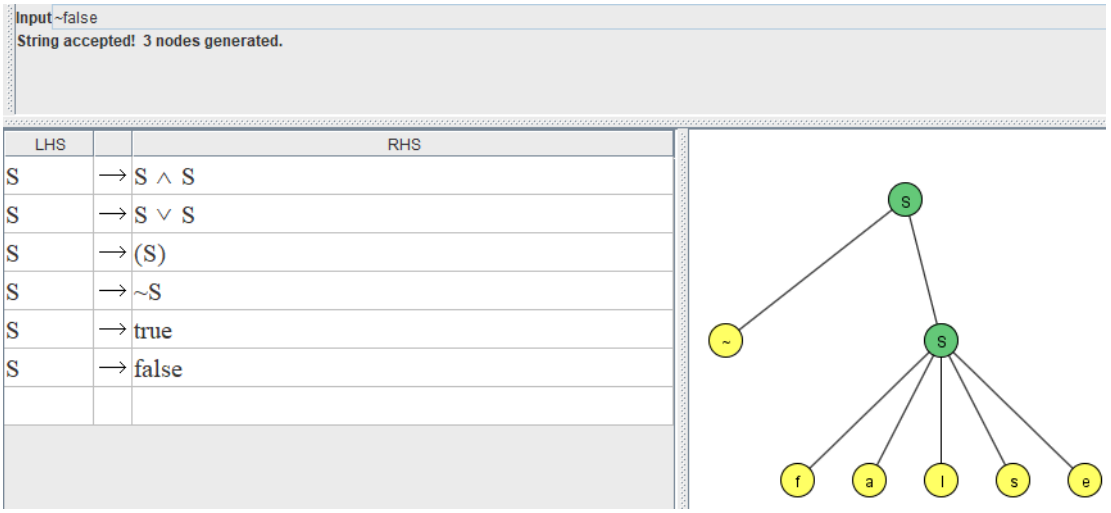
No existe ninguna producción unitaria

3. Eliminación de símbolos y producciones inútiles

Etapa 1:
V’ = ∅
debido a la producción S → true se añade S a V’: V’ = V’ ∪ {S}; V’ = {S}
Como V = V’ no se elimina ningún no terminal
Etapa 2:
J = {S}, V’ = {S}, Σ’ = ∅
Se extrae S de J
Se añaden a Σ’ todos los no terminales de las producciones a partir de S:
Como V’ = V y Σ’ = Σ.
El resultado es el mismo al anterior.







9. **Diseñar una gramática independiente del contexto que genere expresiones aritméticas simples con suma y multiplicación, utilizando los s´ımbolos +, ∗, (,) y los números 0, 1, etc. Ejemplos: “1+2”, “(1+2)*3”, “4*(5+6)”.**

Tengo en cuenta todas las posibilidades como dos llamadas sobre sí mismas de S con la expresión aritmética en medio y otras dos producciones para 0 y 1.

S⟶ S+S | S∗S | (S) | 0S | 1S | 2S | 3S | 4S | 5S | 6S | 7S | 8S | 9S | 0 | 1| 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Input	Result
10+23	Accept
0*0	Accept
(43241)	Accept
10+(4*1)	Accept
9123	Accept
102+	Reject
102+	Reject
*	Reject
(123+*123)	Reject
231*	Reject

A continuación es momento de simplificar la gramática:

S	→	S+S
S	→	S*S
S	→	(S)
S	→	0S
S	→	1S
S	→	2S
S	→	3S
S	→	4S
S	→	5S
S	→	6S
S	→	7S
S	→	8S
S	→	9S
S	→	0
S	→	1
S	→	2
S	→	3
S	→	4
S	→	5
S	→	6
S	→	7
S	→	8
S	→	9

1. Eliminación de producciones vacías:

Como no hay ninguna producción vacía. El resultado del algoritmo es el anterior.

2. Eliminación de producciones unitarias:

No existe ninguna producción unitaria

3. Eliminación de símbolos y producciones inútiles

Etapa 1:

V' = ∅

debido a la producción S → 0 se añade S a V': V' = V' ∪ {S}; V' = {S}

Como V = V' no se elimina ningún no terminal

Etapa 2:

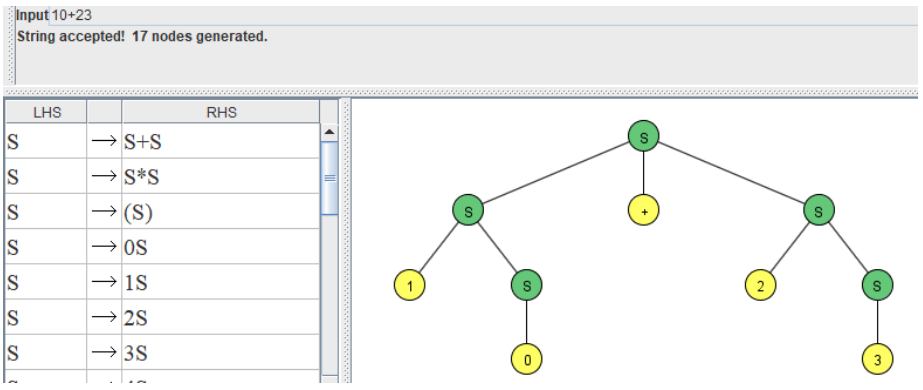
J = {S}, V' = {S}, Σ' = ∅

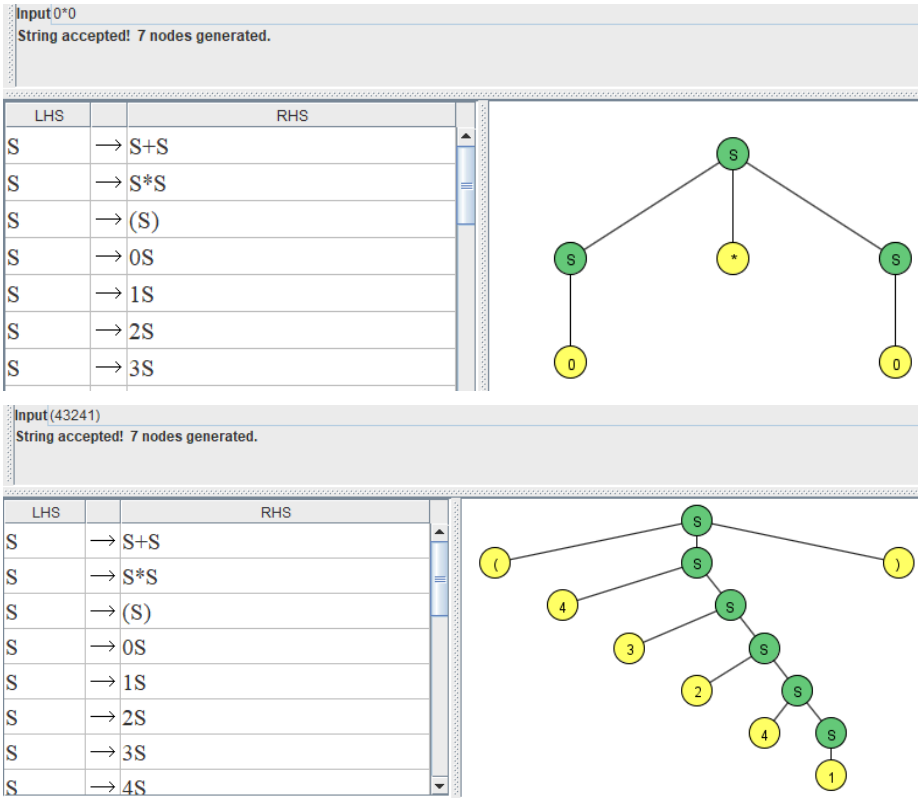
Se extrae S de J

Se añaden a Σ' todos los no terminales de las producciones a partir de S:

Como V' = V y Σ' = Σ.

El resultado es el mismo al anterior.





10. **Diseñar una gramática independiente del contexto que genere el lenguaje de listas anidadas usando corchetes, como en los lenguajes de programación. Ejemplos: [], [[]], [[[]]], [[1,2],[3,4]].**

Es necesario tener en cuenta tres posibilidades para empezar: Que puede ser solo los corchetes, que pueden ser varios corchetes anidados y que existan números en estos corchetes y puedan estar separados por comas. Para cumplir con esto último es necesario usar un no terminal A, que contemple la combinación de símbolos numéricos de cualquier longitud, las opciones mencionadas para el primer no terminal y la posibilidad de usar comas.

S→[] | [S] | [A]
A→A, A | [A] | [] | 0S | 1S | 2S | 3S | 4S | 5S | 6S | 7S | 8S | 9S | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Input	Result
[]	Accept
[1]	Accept
[1, 2]	Accept
[[1], [10]]	Accept
[[], []]	Accept
	Reject
[	Reject
[], []	Reject
[1,2	Reject
[123, 213]]	Reject

A continuación es momento de simplificar la gramática:

4. Eliminación de producciones vacías:

Como no hay ninguna producción vacía. El resultado del algoritmo es el anterior.

5. Eliminación de producciones unitarias:

No existe ninguna producción unitaria

6. Eliminación de símbolos y producciones inútiles

Etapa 1:

V' = ∅

debido a la producción S → [] se añade S a V': V' = V' ∪ {S}; V' = {S}

debido a la producción A → 0 se añade A a V': V' = V' ∪ {A}; V' = {S, A}

Como V = V' no se elimina ningún no terminal

Etapa 2:

J = {S}, V' = {S}, Σ' = ∅

Se extrae S de J

Se añaden a Σ' todos los no terminales de las producciones a partir de S

Se añade A a J

Se repite el bucle extrayendo A y se obtiene:

V' = V y Σ' = Σ.

El resultado es el mismo al anterior.

S	→	[]
S	→	[S]
S	→	[A]
A	→	A, A
A	→	[]
A	→	[A]
A	→	0A
A	→	1A
A	→	2A
A	→	3A
A	→	4A
A	→	5A
A	→	6A
A	→	7A
A	→	8A
A	→	9A
A	→	0
A	→	1
A	→	2
A	→	3
A	→	4
A	→	5
A	→	6
A	→	7
A	→	8
A	→	9

